

Lab #2: Filter Design

Dr. Yang Liu, Dr. Najeeb Najeeb

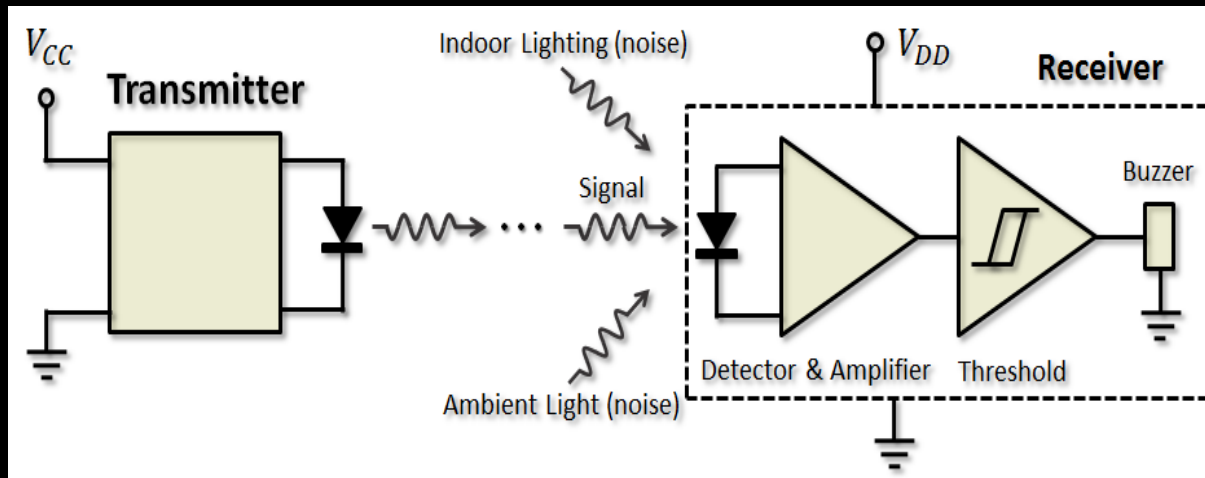
Version
1.2

Logistics for Lab #02

- Lab 2 is due on Friday, April 3
- Progress update with TA on the week of March 23
- Lab checkoff on the week of March 30
- Lab locations
 - 2319 SC
 - 1313 SC
- Lab access:
 - No scheduled lab periods.
 - Lab is available 24/7 (providing that proper lab etiquette is maintained).
- Lab maintenance:
 - Students are in charge of keeping the lab a clean and pleasant working environment.
 - To report problems (e.g., multimeter/oscilloscope malfunction, solder restock, etc.), email TA Nick and cc instructors.

The “Electric Eye”

Light beam interrupter safety system



Design for Uncertainty

- Noisy sensors are common in sensor design.
 - proximity detectors, wireless signal strength, and accelerometers are all notoriously noisy.
- Naïve system design that assumes perfect inputs cannot be trusted.
 - e.g., noisy proximity sensors trigger false alarms
 - The Boy Who Cried Wolf problem.
 - phenomenon of “alarm fatigue” in hospitals.

Example Application: A Garage Door Opener

“The revised standard requires that residential garage door openers contain one of the following:

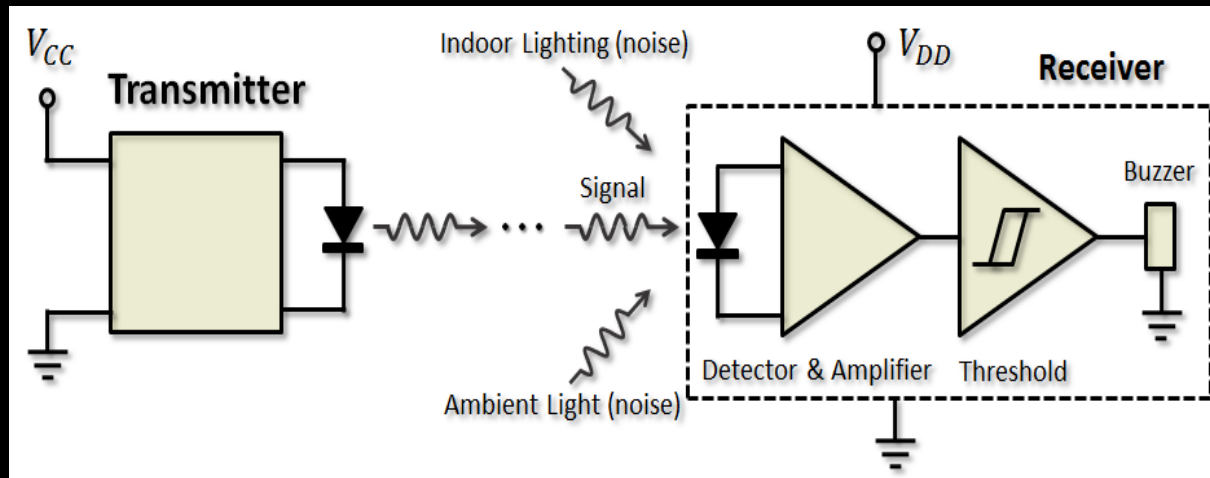
- External entrapment protection device, such as an "electric eye" which "sees" an object obstructing the door [...]"

- 1993 regulation by the Federal Consumer Product Safety Commission (CPSC).



Lab #2: Design a Robust Electric Eye

- A variety of ambient lighting conditions.
 - Exposure to a different transmitter.
- Must separate “signal” from “noise”.



Lab Requirements

- You will be provided with a Tx and a prototype Rx.
 - You will build a receiver to match the provided transmitter.
- Your receiver must work at least as well as the prototype receiver and should be as selective as possible.
 - When exposed to interfering sources, the receiver should continue to function as much as possible.
- **Alert feature.** Your receiver must send text or email alerts when a breach is detected. The format for the alert is specified in the lab spec on ICON.

Requirements (Continued)

- A slight performance degradation (e.g., a slightly increased latency) is acceptable for increased sensitivity/selectivity.
- You are not allowed to modify the transmitter in any way. It is ok to use the existing receiver prototype as a starting point and then modify it.
 - Or start from scratch.
- Your receiver should not have an abnormally large size or weight or other physical characteristics.
 - You can use an “external” processor to generate the alert, but you need to take into consideration the physical/space constraints.

Additional Information

- Schematics for the prototype transmitter and receiver will be available on ICON.
 - You can use these to build a replica of the transmitter for testing.
- HOWEVER, the prototype transmitter may have subtle differences from the schematic.
 - e.g., reported component values are slightly off.
 - (to be clear: the schematic is not deliberately inaccurate).

Test Tour Assumptions!

- You are expected to design and conduct experiments confirming your understanding of the transmitter behavior.
 - Your lab report must contain the design of these experiments and the resulting observations.
- Your receiver must be compatible with the prototype transmitter as it exists.
 - Not the characteristics stated in the schematic.

Design Options

- You need a type of frequency-selective filter.
 - Low-pass, high-pass, band-pass, etc.
- You can build a filtering circuit.
 - Passive or active.
- You can also design a digital filter (software).
 - A template Arduino sketch that implements a digital filter is provided.

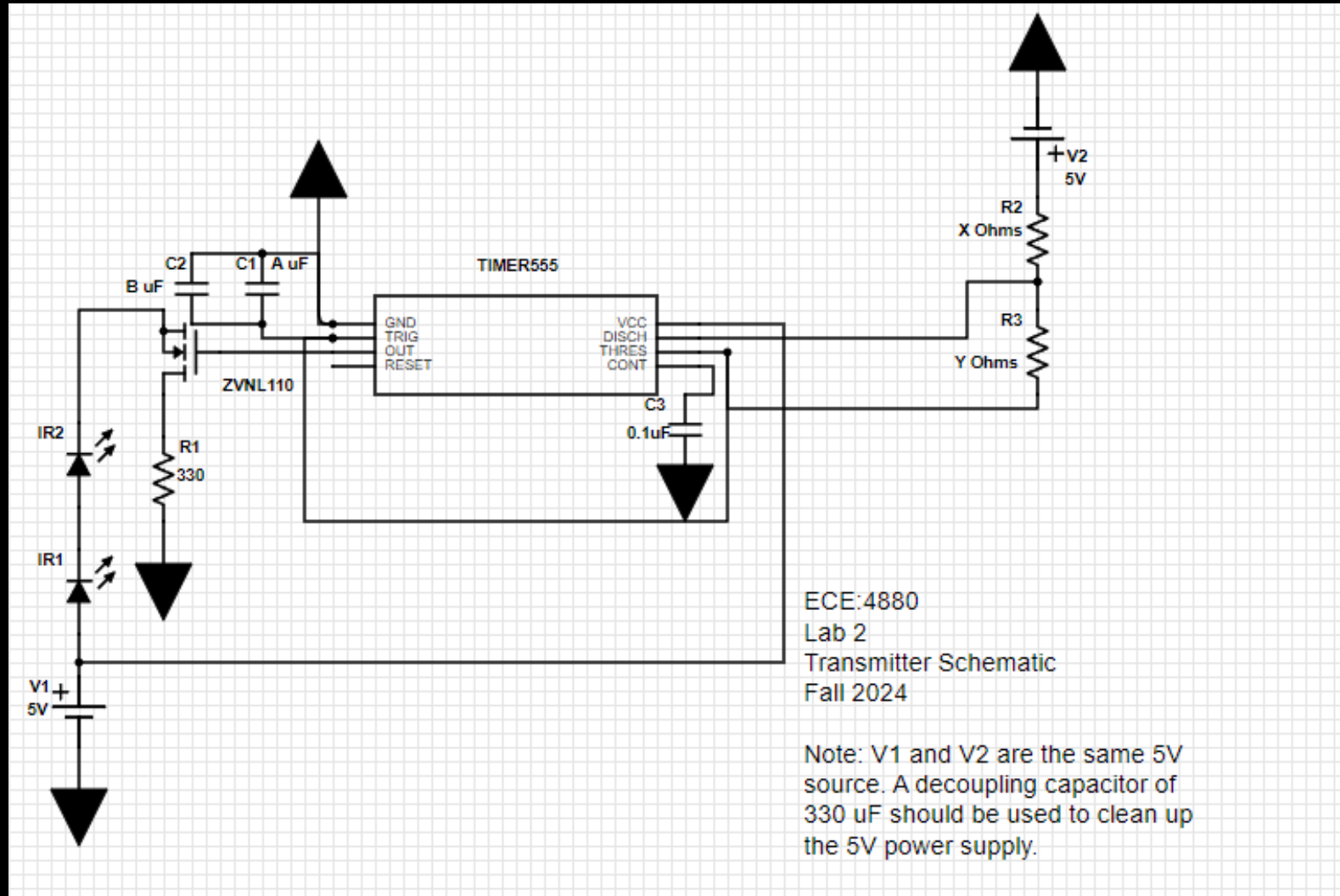
What is Expected From You

- Study the provided transmitter in the lab.
 - Available during TA hours or by appointment.
- Perform experiments or take measurements from the transmitter.
- Create a specification for the filter.
 - Low-pass/band-pass, etc.,
 - What is the cutoff frequency, etc.

What is NOT Expected From You

- You are NOT expected to design or build filters.
 - Standard tools are available, and you can seek help with that.
- You are NOT expected to explain how your filtering circuit or sketch works.

Transmitter Schematic



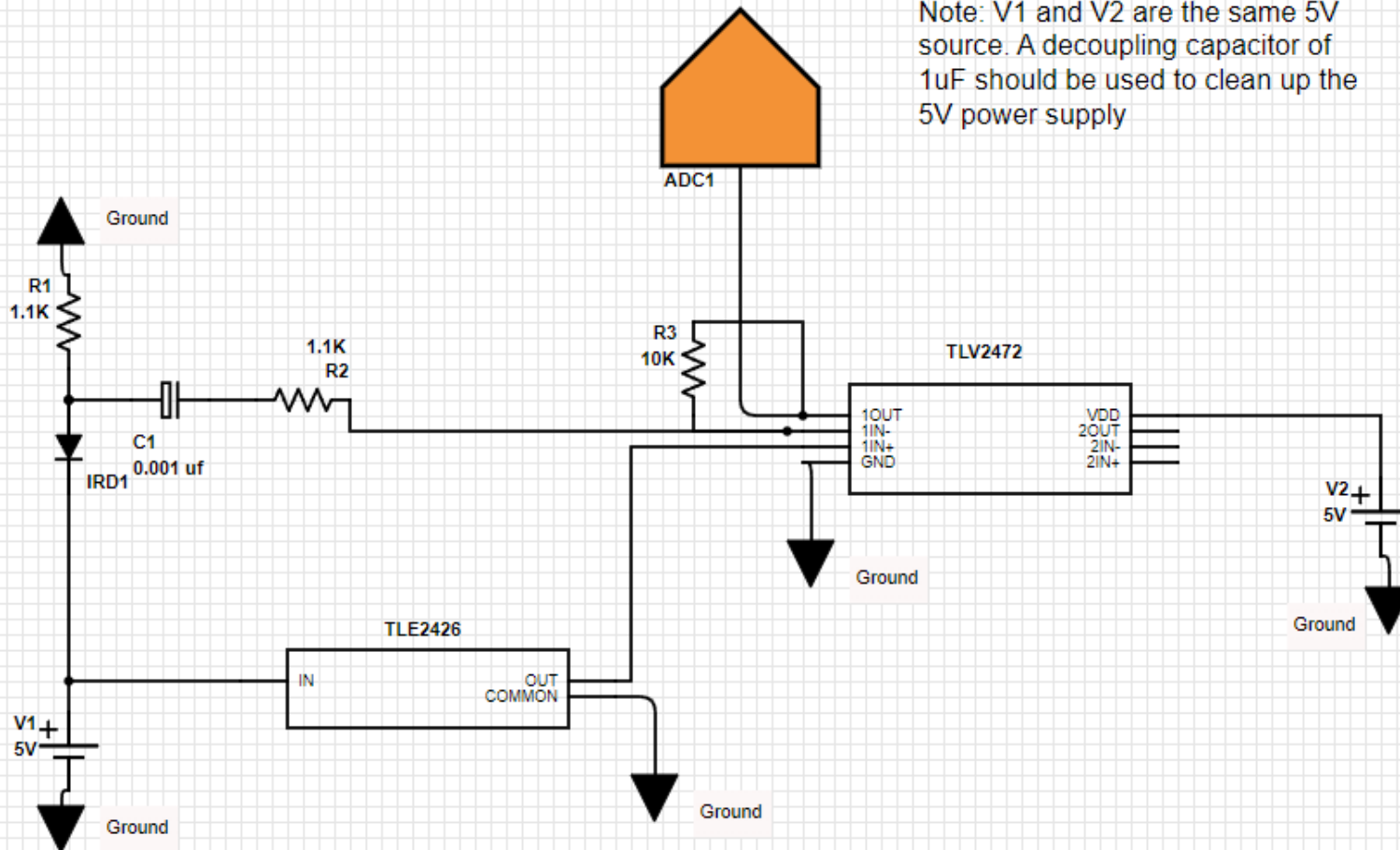
Receiver (front-end) Schematic

Receiver Circuit Schematic

ECE:4880 Fall 2024

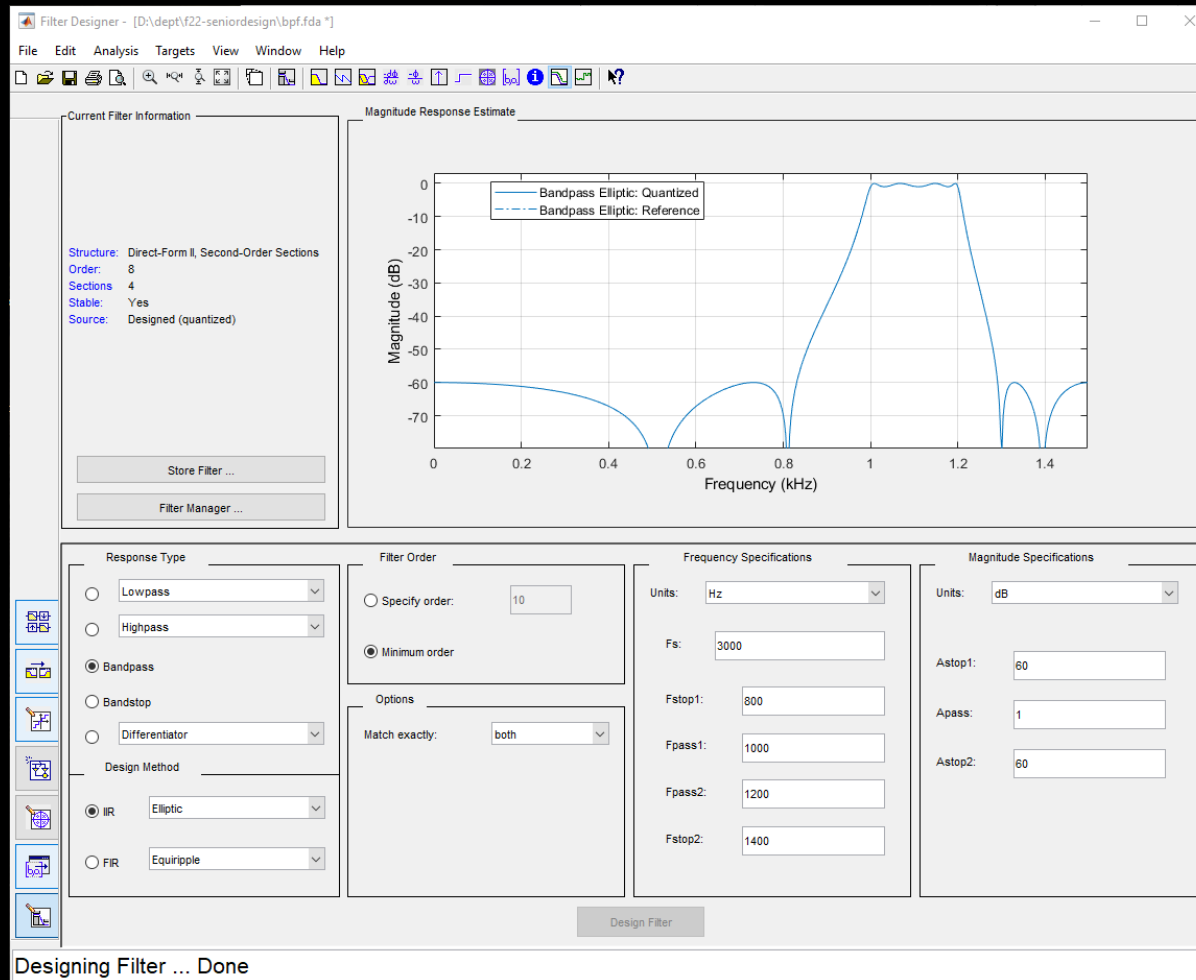
LAB 2

Note: V1 and V2 are the same 5V source. A decoupling capacitor of 1uF should be used to clean up the 5V power supply



Filter Design Tool example

- A MATLAB-based GUI tool



Arduino Sketch

```
const int n = 7; // number of past input and output samples to buffer; change this to match order of your filter
int m = 10; // number of past outputs to average for hysteresis

float den[] = {1.0000, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00}; //Denominator Coefficients
float num[] = {0.0058, 0.00, -0.015, 0.00, 0.015, 0.00, -0.0058}; //Numerator Coefficients

float x[n], y[n], yn, s[10]; // Space to hold previous samples and outputs; n'th order filter will require upto n s
float threshold_val = 0.2; // Threshold value. Anything higher than the threshold will turn the LED off. Anything low
// time between samples Ts = 1/Fs. If Fs =
int Ts = 333;

void setup()
{
  Serial.begin(1200);
  int i;

  sbi(ADCSRA, ADPS2); // Next three 1:
  cbi(ADCSRA, ADPS1);
  cbi(ADCSRA, ADPS0);

  pinMode(LED, OUTPUT); // Makes the LED

  for (i=0; i<n; i++)
  {
    x[i] = y[i] = 0;
  }

  for (i = 0; i<m; i++)
  {
    s[i] = 0;
    yn = 0;
  }

  yn = num[0] * x[0];

  for (i=1; i<n; i++) // Incorporate previous outputs (y[n])
  {
    yn = yn - den[i] * y[i] + num[i] * x[i];
  }

  y[0] = yn; // New output

  // The variable yn is the output of the filter at this time step.
  // Now we can use it for its intended purpose:
  // - Apply threshold
  // - Apply hysteresis
  // - What to do when the beam is interrupted, turn on a buzzer, send SMS alert.
  // - etc.

  s[0] = abs(2*yn); // Absolute value of the filter output.

  [.....]

  // The filter was designed for a 3000 Hz sampling rate. This corresponds
  // to a sample every 333 us. The code above must execute in less time
  // (if it doesn't, it is not possible to do this filtering on this processor).
  // Below we tread some water until it is time to process the next sample

  if ((micros() - t1) > Ts)
  {
    // if this happens, you must reduce Fs, and/or simplify your filter to run faster
    Serial.println("MISSED A SAMPLE");
  }
  while ((micros() - t1) < Ts);
}
```